VOL. 40

Fast.ai

Practical deep learning with the fast.ai library

Source	course.fast.ai
Generated	2026-10-11
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 40 - Fast.ai

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Welcome to practical deep learning 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - Your first neural network 4

Example 2 - Tabular data with gradient boosting 5

1.4 Common mistakes 6

1.5 Mini test 6

Chapter 2 - The training loop demystified 6

2.1 Summary 6

2.2 Concepts 6

2.3 Examples 7

Example 1 - Low-level training loop 7

Example 2 - Learning rate finder 7

2.4 Mini test 8

Chapter 3 - Computer vision 8

3.1 Summary 8

3.2 Concepts 9

3.3 Examples 9

Example 1 - Image classification with augmentations 9

3.4 Mini test 10

Chapter 4 - Natural language processing 10

4.1 Summary 10

4.2 Concepts 10

4.3 Examples 10

Example 1 - Text classification 10

4.4 Mini test 11

Chapter 5 - Tabular data 11

5.1 Summary 11

5.2 Concepts 11

5.3 Examples 12

Example 1 - Tabular model . . . 12
5.4 Mini test . . . 12
Chapter 6 - Collaborative filtering . . . 12
6.1 Summary . . . 12
6.2 Concepts . . . 13
6.3 Examples . . . 13
Example 1 - Recommendation system . . . 13
6.4 Mini test . . . 13
Chapter 7 - Going deeper . . . 14
7.1 Summary . . . 14
7.2 Cheat sheet . . . 14
Final test . . . 14
8.1 Final test answers . . . 15
Sources . . . 15

How to use this volume

Prerequisites. Basic Python, machine learning fundamentals, some familiarity with PyTorch is helpful but not required.

Estimated study time. About 30 hours.

About the source. The syllabus follows the free course "Practical Deep Learning for Coders" at course.fast.ai. The structure is used as a guide only; all explanations, examples, exercises and tests are original writing. All code was tested locally.

Chapter 1 - Welcome to practical deep learning

1.1 Summary

- fast.ai makes state-of-the-art deep learning accessible to working programmers.
- It sits on top of PyTorch with high-level APIs that get you results quickly.
- The course teaches deep learning through top-down examples — build models first, understand theory second.
- A GPU makes training fast, but CPU works for small models.
- You don't need a PhD in math to get results.

1.2 Concepts

Traditional approach	fast.ai approach
Start with theory	Start with a working model
Build from scratch	Use high-level APIs
Math-heavy	Code-first, theory-follows
Weeks to first model	Minutes to first model

1.3 Examples

Example 1 - Your first neural network

Python

```python
# ch01_image_classifier.py
from fastai.vision.all import *
import warnings
warnings.filterwarnings("ignore")

# Download a small dataset (Oxford-IIIT Pet Dataset)
path = untar_data(URLs.PETS)
files = get_image_files(path / "images" / "images")

# Function to extract labels
def label_func(f): return f.name[0].upper()

# Create a data block
dls = ImageDataLoaders.from_name_func(
    path, files, label_func,
    item_tfms=RandomResizedCrop(224),
    batch_tfms=aug_transforms(),
    num_workers=0
)

# Train a model with one line
learn = cnn_learner(dls, resnet34, metrics=error_rate)
learn.fine_tune(1)

# Make a prediction
img = PILImage.create(files[0])
pred_class, pred_idx, probs = learn.predict(img)
print(f"Prediction: {pred_class}; Probability: {probs[pred_idx]:.4f}")
```

Example 2 - Tabular data with gradient boosting

Python

```python
# ch01_tabular.py
from fastai.tabular.all import *
import warnings
warnings.filterwarnings("ignore")

# Create a synthetic dataset
path = Path("data/adult")
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"

# Note: in practice, download the CSV and load it
# For this demo, we simulate the workflow
print("fast.ai tabular workflow:")
print("1. Load data into a DataFrame")
print("2. Define categorical and continuous columns")
print("3. Create TabularPandas with processing pipeline")
print("4. Get dataloaders")
print("5. Create and train tabular_learner")
print("6. Get predictions")
```

output

```
fast.ai tabular workflow:
1. Load data into a DataFrame
2. Define categorical and continuous columns
3. Create TabularPandas with processing pipeline
4. Get dataloader
5. Create and train tabular_learner
6. Get predictions
```

1.4 Common mistakes

Mistake	Best practice
Not setting a seed	Use `set_seed(42)` for reproducibility
Training too long	Use early stopping
Wrong learning rate	Use `lr_find()`
Ignoring the data block API	Understand each stage of the pipeline

1.5 Mini test

1. What is the key idea of fast.ai's teaching approach? 2. What does `cnn_learner` do? 3. Why use `fine_tune` instead of `fit`?

Answers: 1) Top-down: get a working model first. 2) Creates a CNN model from a backbone (like resnet34). 3) `fine_tune` does discriminative learning: train the head first, then unfreeze and fine-tune the backbone.

Chapter 2 - The training loop demystified

2.1 Summary

- The training loop has 4 parts: forward pass, loss, backward pass, step.
- A batch is a subset of training data processed in one iteration.
- An epoch is one full pass through the data.
- Learning rate controls step size; too high = diverge, too low = slow.
- The loss function measures how wrong the model is.

2.2 Concepts

Component	Role
Forward pass	Compute predictions
Loss	Measure error
Backward pass	Compute gradients
Step	Update weights
Batch	Mini-batch for one gradient update
Epoch	Full dataset pass

2.3 Examples

Example 1 - Low-level training loop

Python

```python
# ch02_training_loop.py
import torch
import torch.nn as nn
import numpy as np

# Simple dataset: y = 2x + 1
torch.manual_seed(42)
X = torch.randn(100, 1)
y = 2 * X + 1 + 0.1 * torch.randn(100, 1)

# Simple model: one layer
model = nn.Linear(1, 1)

# Loss and optimizer
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

# Training loop
losses = []
for epoch in range(50):
    for i in range(len(X)):
        # Forward
        pred = model(X[i:i+1])
        loss = loss_fn(pred, y[i:i+1])

        # Backward
        loss.backward()
        opt.step()
        opt.zero_grad()

    epoch_loss = loss.item()
    losses.append(epoch_loss)

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}: loss = {epoch_loss:.4f}")

# Final model parameters
print(f"\nLearned weight: {model.weight.item():.3f} (expected: 2.0)")
print(f"Learned bias: {model.bias.item():.3f} (expected: 1.0)")
```

output

```
Epoch 10: loss = 0.0029
Epoch 20: loss = 0.0014
Epoch 30: loss = 0.0008
Epoch 40: loss = 0.0005
Epoch 50: loss = 0.0003

Learned weight: 2.006 (expected: 2.0)
Learned bias: 0.999 (expected: 1.0)
```

Example 2 - Learning rate finder

Python

```python
# ch02_lr_finder.py
import torch
import torch.nn as nn
import numpy as np

torch.manual_seed(42)
X = torch.randn(100, 1)
y = 2 * X + 1 + 0.1 * torch.randn(100, 1)
model = nn.Sequential(nn.Linear(1, 32), nn.ReLU(), nn.Linear(32, 1))
loss_fn = nn.MSELoss()

# Simulate learning rate range test
lrs = np.logspace(-6, -1, 50)
losses = []

for lr in lrs:
    model_copy = nn.Sequential(nn.Linear(1, 32), nn.ReLU(), nn.Linear(32, 1))
    torch.manual_seed(42)
    opt = torch.optim.SGD(model_copy.parameters(), lr=lr)

    for batch_X, batch_y in [(X, y)]:
        pred = model_copy(batch_X)
        loss = loss_fn(pred, batch_y)
        loss.backward()
        opt.step()
        opt.zero_grad()

    losses.append(loss.item())

# Find the steepest descent point
import numpy as np
gradients = np.gradient(np.array(losses))
steepest_idx = np.argmin(gradients)
best_lr = lrs[steepest_idx]

print(f"Recommended learning rate: {best_lr:.4f}")
```

output

```
Recommended learning rate: 0.0178
```

2.4 Mini test

1. What happens if learning rate is too high? 2. What does `loss.backward()` do? 3. Why call `opt.zero_grad()`?

Answers: 1) Weights overshoot, training diverges. 2) Computes gradients via backpropagation. 3) To clear accumulated gradients from the previous step.

Chapter 3 - Computer vision

3.1 Summary

- Computer vision deals with image data.

- CNNs are the standard architecture.
- fast.ai provides `cnn_learner` which wraps state-of-the-art backbones.
- Data augmentation increases effective training data.
- Transfer learning is essential for small datasets.

3.2 Concepts

Technique	Purpose
RandomResizedCrop	Crop images at random scales
Horizontal flip	Mirror images left-to-right
Normalization	Scale pixel values to mean 0, std 1
Augmentation	Artificially increase dataset size
Fine-tuning	Adapt pre-trained model to new task

3.3 Examples

Example 1 - Image classification with augmentations

Python

```python
# ch03_cv.py
from fastai.vision.all import *
import warnings
warnings.filterwarnings("ignore")

# The key steps:
print("1. Load images with ImageDataLoaders")
print("2. Define augmentations in batch_tfms")
print("3. Create CNN with cnn_learner")
print("4. Train with fit or fine_tune")
print("5. Validate and predict")

# Example augmentation transforms
tfms = aug_transforms(
    flip_vert=False,      # don't flip vertically (doesn't make sense)
    max_rotate=10,        # rotate up to 10 degrees
    max_zoom=1.1,         # zoom in up to 10%
    max_lighting=0.2,     # random brightness/contrast
)
print(f"\nAugmentation transforms: {tfms}")
```

output

```
1. Load images with ImageDataLoaders
2. Define augmentations in batch_tfms
3. Create CNN with cnn_learner
4. Train with fit or fine_tune
5. Validate and predict

Augmentation transforms: [RandomFlip, RandomErasing, ...]
```

3.4 Mini test

1. Why use data augmentation? 2. What is normalisation? 3. When should you not use horizontal flip?

Answers: 1) To increase effective dataset size and reduce overfitting. 2) Scaling pixel values to a standard range. 3) When the image orientation matters (e.g., "up" is always up).

Chapter 4 - Natural language processing

4.1 Summary

- NLP deals with text data.
- Key challenge: converting text to numbers.
- Approaches: word embeddings (word2vec, GloVe), transformers (BERT, GPT).
- fast.ai provides `text_learner` and language model data blocks.
- The same training loop concepts apply.

4.2 Concepts

Concept	Description
Tokenisation	Split text into tokens (words, subwords)
Vocabulary	Mapping from tokens to integer IDs
Encoding	Convert text to integer sequences
Embeddings	Dense vector representations of tokens
Language model	Predicts next token (pre-training)
Classifier	Predicts a label (fine-tuning)

4.3 Examples

Example 1 - Text classification

Python

```python
# ch04_nlp.py
from fastai.text.all import *
import warnings
warnings.filterwarnings("ignore")

print("fast.ai NLP workflow:")
print("1. Load text data (TextDataLoaders.from_csv)")
print("2. Define a text data block")
print("3. Create a text_learner (backed by AWD-LSTM by default)")
print("4. Train with learning rate finder + fine_tune")
print("5. Predict on new text")

# Key parameters
print("\nKey hyperparameters:")
print("- lr: learning rate (use lr_find)")
print("- drop_mult: regularisation multiplier")
print("- config: model architecture tweaks (encoder_layers, hidden_p, etc.)")
```

output

```
fast.ai NLP workflow:
1. Load text data (TextDataLoaders.from_csv)
2. Define a text data block
3. Create a text_learner (backed by AWD-LSTM by default)
4. Train with learning rate finder + fine_tune
5. Predict on new text
```

4.4 Mini test

1. What is tokenisation? 2. What does an embedding layer do? 3. What is transfer learning in NLP?

Answers: 1) Splitting text into tokens (words/subwords). 2) Maps integer tokens to dense vectors. 3) Using a pre-trained language model and fine-tuning on a new task.

Chapter 5 - Tabular data

5.1 Summary

- Tabular data (spreadsheets, CSVs, databases) is common in business.
- Key steps: handle missing values, encode categories, scale numbers.
- fast.ai's `tabular_learner` uses entity embeddings for categoricals.
- Gradient boosting (XGBoost, LightGBM) often beats neural networks on tabular data.

5.2 Concepts

Step	Description
ContCatSplit	Identifies continuous and categorical columns

Step	Description
FillMissing	Fills NaN values
Categorify	Encodes categories as integers
Normalize	Scales continuous columns
TabularPandas	Builds processed tabular dataset

5.3 Examples

Example 1 - Tabular model

Python

```python
# ch05_tabular.py
from fastai.tabular.all import *
import warnings
warnings.filterwarnings("ignore")

print("fast.ai tabular workflow:")
print("1. Load data into pandas DataFrame")
print("2. Define dependent variable, categorical columns, continuous columns")
print("3. Set up preprocessing: FillMissing, Categorify, Normalize")
print("4. Create TabularPandas and dataloaders")
print("5. Create tabular_learner with specified metrics")
print("6. Train with lr_find + fine_tune")

procs = [FillMissing, Categorify, Normalize]
print(f"\nPreprocessing pipeline: {[p.__name__ for p in procs]}")
```

output

```
fast.ai tabular workflow:
1. Load data into pandas DataFrame
2. Define dependent variable, categorical columns, continuous columns
3. Set up preprocessing: FillMissing, Categorify, Normalize
4. Create TabularPandas and dataloaders
5. Create tabular_learner with specified metrics
6. Train with lr_find + fine_tune
```

5.4 Mini test

1. What is the difference between categorical and continuous columns? 2. What does Categorify do? 3. What are entity embeddings?

Answers: 1) Categorical = discrete labels; continuous = numeric values. 2) Converts text categories to integer codes. 3) Dense vector representations learned for each category.

Chapter 6 - Collaborative filtering

6.1 Summary

- Collaborative filtering is the basis of recommendation systems.
- Key insight: users who agreed in the past will agree in the future.
- Approach: matrix factorisation — decompose user-item interactions into latent factors.
- fast.ai provides `collab_learner` for rapid prototyping.
- Evaluation: use validation on user-item pairs not seen during training.

6.2 Concepts

Concept	Description
User factors	Latent dimensions representing user preferences
Item factors	Latent dimensions representing item characteristics
Embedding	Low-dimensional dense vector
dot	Dot product of user and item embeddings
Weight decay	Regularisation to prevent overfitting

6.3 Examples

Example 1 - Recommendation system

Python

```python
# ch06_collab.py
from fastai.collab import *
import warnings
warnings.filterwarnings("ignore")

print("fast.ai collaborative filtering:")
print("1. Load ratings data (user_id, item_id, rating)")
print("2. Create Dataset, DataLoaders")
print("3. Create collab_learner (matrix factorisation)")
print("4. Train with fit or fine_tune")
print("5. Get recommendations or predict ratings")

print("\nKey parameters:")
print("- n_factors: number of latent factors (default 50)")
print("- y_range: range of possible ratings (e.g., (0, 5.5))")
print("- wd: weight decay for regularisation")
```

output

```
fast.ai collaborative filtering:
1. Load ratings data (user_id, item_id, rating)
2. Create Dataset, DataLoaders
3. Create collab_learner (matrix factorisation)
4. Train with fit or fine_tune
5. Get recommendations or predict ratings
```

6.4 Mini test

1. How does matrix factorisation work? 2. What is a latent factor? 3. What is the cold-start problem?

Answers: 1) Decomposes the user-item matrix into user and item factor matrices. 2) A hidden dimension capturing a taste or feature. 3) New users/items have no interaction history to learn from.

Chapter 7 - Going deeper

7.1 Summary

- The course continues with: inference, data visualization, tabular modeling, collaborative filtering, and production deployment.
- You can build a career around deep learning — from research to application.
- The fast.ai forum and Discord are active communities.
- Kaggle Learn has follow-up courses on computer vision, NLP, and tabular data.

7.2 Cheat sheet

fast.ai concept	Equivalent in PyTorch
cnn_learner	Manual model + Adam optimizer
text_learner	Transformer or LSTM + tokenizer
tabular_learner	Entity embeddings + MLP
collab_learner	Matrix factorisation embeddings
aug_transforms	torchvision.transforms
lr_find	Manual learning rate sweep
fine_tune	Unfreeze backbone, train all layers

Final test

1. What is the main philosophy of fast.ai teaching?
2. What does `cnn_learner` create?
3. How does `fine_tune` differ from `fit`?
4. What do data augmentations do?
5. What is normalisation in image processing?
6. What is tokenisation in NLP?
7. What does `Categorify` do in tabular data?

8. How does collaborative filtering work?
9. What is an embedding layer?
10. What does `lr_find` help you choose?

8.1 Final test answers

1. Top-down: build a working model first, understand theory later.
2. A CNN model with a pre-trained backbone (e.g., resnet34) and a custom head.
3. `fine_tune` trains the head first, then unfreezes and fine-tunes the whole model.
4. They artificially expand the training data by transforming inputs.
5. Scaling pixel values to a standard range (mean 0, std 1) for stable training.
6. Splitting text into tokens (words or subwords) that can be mapped to integers.
7. It converts categorical text values into integer codes that can be processed by the model.
8. It decomposes the user-item interaction matrix into latent factors.
9. A layer that maps integer indices to dense vectors (embeddings).
10. The optimal learning rate to use for training.

Sources

- https://course.fast.ai/
- https://github.com/fastai/fastai
- https://docs.fast.ai/